

# Post-Quantum SSH Hardening in 2026: The Transition to mlkem768x25519-sha256 in OpenSSH

Day 03 | Category: Network Protocol Hardening

Date: 2026-07-10 |

DAILY MONITORING

In the transition to post-quantum resistance, securing administrative channels is just as crucial as protecting web traffic. **Secure Shell (SSH)**, the backbone of modern cloud and server administration, has seen rapid PQC integration. While previous versions relied on experimental hybrid options (like `sntrup761x25519-sha512`), modern implementations like **OpenSSH 9.9 (and newer)** and platforms like **Ubuntu 26.04 LTS** have established `mlkem768x25519-sha256` as the preferred default key exchange (KEX) mechanism.

**Underlying Security Principle:** The key exchange algorithm `mlkem768x25519-sha256` is a hybrid mechanism. It combines **ML-KEM-768** (the NIST FIPS 203 module-lattice-based key encapsulation mechanism) with the classical **X25519** elliptic curve Diffie-Hellman (ECDH), binding them using a SHA-256-based KDF. This "hybrid" design ensures that if ML-KEM is ever broken by future mathematical breakthroughs, the connection remains as secure as standard X25519.

## The Evolution of PQ-KEX in OpenSSH

To understand the current state, we must look at how OpenSSH managed its post-quantum transition:

- **OpenSSH 8.5 to 9.8:** Introduced and defaulted to `sntrup761x25519` (combining Streamlined NTRU Prime 761 and X25519) because NTRU Prime was considered highly conservative and immune to certain patent concerns. However, it was not the final NIST standard.
- **OpenSSH 9.9:** Following the official publication of FIPS 203 by NIST in late 2024, OpenSSH introduced `mlkem768x25519-sha256` and made it the highest-priority key exchange algorithm, while deprecating and removing obsolete algorithms like DSA.
- **Ubuntu 26.04 LTS (2026):** Out of the box, Ubuntu 26.04's SSH client and server negotiate `mlkem768x25519-sha256` by default, ensuring that every administrative session is quantum-safe from day one.

## Configuring and Hardening OpenSSH for PQC

By default, OpenSSH negotiates the strongest mutually supported algorithm. However, to comply with strict post-quantum requirements and eliminate classical fallback risks, administrators can enforce a post-quantum-only key exchange policy.

## Auditing the Negotiated Algorithm

To inspect what key exchange algorithm was negotiated for an active session, connect using double-verbose mode (-vv) and grep for the key exchange (kex) negotiations:

```
ssh -vv user@server exit 2>&1 | grep "kex:"
```

If the client and server are updated, you will see:

```
debug1:  kex:  algorithm:  mlkem768x25519-sha256
```

## Enforcing Post-Quantum-Only KEX

To restrict your server to only accept post-quantum and hybrid key exchanges, edit your SSH daemon configuration (e.g., in /etc/ssh/sshd\_config.d/99-pqc.conf):

### Configuration Snippet:

```
# Force post-quantum hybrid key exchange only
KexAlgorithms mlkem768x25519-sha256,sntrup761x25519-sha512

# Enable verbose logging to audit client connection capabilities
LogLevel VERBOSE
```

Before restarting the daemon, always validate the syntax to avoid locking yourself out: `sudo sshd -t`. Once verified, restart with `sudo systemctl restart ssh`.

## Algorithm & Performance Comparison in SSH

The following table outlines the key exchange algorithms available in modern SSH daemons and their corresponding operational metrics:

| Algorithm Name         | Type           | Key Exchange Overhead (Bytes) | Cryptographic Foundation        | Quantum-Resistant? | Production Status (2026) |
|------------------------|----------------|-------------------------------|---------------------------------|--------------------|--------------------------|
| curve25519-sha256      | Classical ECDH | 64                            | Curve25519 (Discrete Logarithm) | No                 | Fallback Only            |
| sntrup761x25519-sha512 | Hybrid KEM     | ~1,200                        | NTRU Prime + X25519             | Yes (Hybrid)       | Supported (Legacy PQ)    |

| Algorithm Name        | Type       | Key Exchange Overhead (Bytes) | Cryptographic Foundation    | Quantum-Resistant? | Production Status (2026) |
|-----------------------|------------|-------------------------------|-----------------------------|--------------------|--------------------------|
| mlkem768x25519-sha256 | Hybrid KEM | ~2,300                        | ML-KEM-768 (M-LWE) + X25519 | Yes (Hybrid)       | Default Standard         |

## Daily Quiz: Test Your Understanding

1. Why is `mlkem768x25519-sha256` preferred over `sntrup761x25519-sha512` in OpenSSH 9.9 and modern operating systems like Ubuntu 26.04?

A) Because NTRU Prime was found to be completely broken by classical linear computers.  
B) Because ML-KEM-768 is the official NIST standard (FIPS 203), whereas NTRU Prime was not selected by NIST.  
C) Because ML-KEM-768 has significantly smaller public keys than `curve25519`.

**Correct Answer: B.** Although NTRU Prime is a secure and highly respected scheme (and was used as a stopgap measure), ML-KEM is the standardized post-quantum algorithm defined in NIST's FIPS 203, making it the globally accepted compliance baseline.

2. What is the primary risk of not configuring a post-quantum key exchange on an SSH server today?

A) Attackers can instantly hack the SSH session using modern quantum computers available in 2026.  
B) Legacy SSH clients will not be able to connect to the server anymore.  
C) Sessions are vulnerable to "Harvest Now, Decrypt Later" (HNDL) attacks, where encrypted administrative sessions are recorded now and decrypted once a cryptographically relevant quantum computer is available.

**Correct Answer: C.** Symmetric session keys negotiated using classical Diffie-Hellman or ECDH are vulnerable to retroactive decryption once Shor's algorithm can be run on a quantum computer.

3. What command is used to safely verify the SSH server configuration after hardening it with PQC-only algorithms?

A) `sudo systemctl restart ssh`  
B) `sudo sshd -t`  
C) `ssh-keygen -A`

**Correct Answer: B.** `sudo sshd -t` tests the configuration files for syntax errors without restarting the SSH daemon. Running this command prevents locking yourself out of a remote server due to a typo.